

설비 고장을 예측하는 AI 데이터 분석

시간 연속성 기반 전처리

누락 시점 처리와 전처리 파이프라인

C O N T E N T S

목차

01 채우기와 보간 시계열 결측의 특성 · 보간이란 무엇인가

02 전처리 파이프라인 리샘플링과 결측처리 순서 · 품질 점검

01

채우기와 보간

시계열 결측의 특성, 보간이란 무엇인가

누락이 흔드는 시간 연속성

빈 칸은 값 하나가 아니라 시간 흐름의 단절

추세·변화량·이동평균 계산을 동시에 왜곡

분석 영향

평균·변화량 계산 주변부 왜곡

정비 영향

고장 징후 탐지와 정비 시점 판단 흔들림

일반 결측과 시계열 결측

01

일반 데이터 결측

행 순서 영향 작음 · 전체 평균 대체 허용

02

시계열 데이터 결측

앞뒤 값과 시간 간격이 핵심 추정 단서

전체 평균보다 주변 시점 기반 복원이 더 자연스러운 선택

산발 누락과 연속 누락

구분	산발 누락	연속 누락
모습	한 개씩 흩어진 빈 칸	긴 구간 통째 공백
앞뒤 값 거리	가까운 실제 값	멀리 떨어진 실제 값
복원 신뢰도	상대적으로 높음	상대적으로 낮음
처리 방향	보간 중심 복원	제한 보관 또는 공백 유지

빈 칸 개수보다 분포 패턴이 처리 전략의 기준

dropna 제거의 위험

빈 행 삭제로 다시 불규칙해지는 시간 간격

사라진 시점 때문에 이동평균·변화량 계산 오류

빈 칸이 사라져도 시간 축 품질은 오히려 악화

결측 처리 방법의 두 갈래

01

값 복사

ffill · bfill



02

값 계산

선형 보간 · 시간 보간



03

선택 기준

값의 변화 양상과 결측 길이

ffill 앞값 채우기

CODE

```
# 10초 정규화로 빈 칸이 생긴 제어출력 데이터
norm = df[["제어출력"]].asfreq("10s")
print("ffill 전 빈 칸:", int(norm["제어출력"].isna().sum())) # 출력: 22
# 앞값으로 채우기
filled = norm["제어출력"].ffill()
print("ffill 후 빈 칸:", int(filled.isna().sum())) # 출력: 0
```

ffill과 bfill 방향 비교

구분	ffill	bfill
가져오는 값	앞쪽 마지막 실제 값	뒤쪽 첫 실제 값
채우는 방향	위에서 아래	아래에서 위
남는 빈 칸	맨 앞 공백	맨 뒤 공백

두 방법은 방향만 반대인 값 복사 전략

ffill·bfill의 적합 조건

01 · 상태 값

설정 소입로온도·가동 모드처럼 유지되는 값

02 · 짧은 공백

한두 칸 공백처럼 변화 가능성이 낮은 구간

03 · 빠른 정리

정교한 추정보다 간단한 처리 우선 상황

ffill이 감추는 추세

01

실제 흐름

제어출력·소입로온도 값이 시간에 따라 서서히 상승

02

ffill 결과

빈 구간이 평평한 계단으로 변형

변화 구간을 멈춘 값처럼 보이게 만드는 복사 방식의 한계

실습 1. ffill과 bfill 비교

빈 칸을 앞값·뒷값으로 채우고 조합해 남는 결측 확인

목표

빈 칸을 앞값·뒷값으로 채우고 조합해 남는 결측을 확인

단계

- 정규화 데이터의 제어출력을 앞값으로 채우기
- 같은 데이터를 뒷값으로 채우기
- 두 방법을 이어 붙여 맨 앞뒤 빈 칸까지 채우기

예상 결과

앞값·뒷값 각각 빈 칸 0, 조합도 0

보간의 기본 개념

앞뒤 값을 다리처럼 이어 빈 구간 값을 추정

한쪽 복사가 아니라 양쪽 실제 값을 함께 반영

핵심 가정

값이 비교적 매끄럽게 변화

적합 데이터

제어출력·소입로온도처럼 연속적으로 변하는 센서

선형 보간 interpolate

CODE

```
norm = df[["제어출력"]].asfreq("10s")
print("보간 전 빈 칸:", int(norm["제어출력"].isna().sum())) # 출력: 22
# 선형 보간 (기본값)
lin = norm["제어출력"].interpolate()
print("선형 보간 후 빈 칸:", int(lin.isna().sum())) # 출력: 0
```

시간 기반 보간 method time

CODE

```
import pandas as pd
import numpy as np
# 불규칙 간격 예시: 앞에서 10초, 뒤까지 50초 떨어진 빈 칸
s = pd.Series([10.0, np.nan, 40.0],
               index=pd.to_datetime(["09:00:00", "09:00:10", "09:01:00"]))
print("선형 보간:", s.interpolate(method="linear").iloc[1]) # 출력: 25.0
print("시간 보간:", s.interpolate(method="time").iloc[1])   # 출력: 15.0
# 선형은 칸 위치(중간)로 25, 시간은 실제 간격 비율로 15
```


선형 보간과 시간 보간

구분	선형 보간	시간 보간
기준	빈 칸 개수	실제 시간 간격
등간격 데이터	동일 간격 기준	거의 같은 결과
불규칙 간격	시간 왜곡 위험	간격 비율 반영
조건	별도 조건 낮음	시간 인덱스 요구

정규화된 등간격 데이터는 선형 보간도 안정적인 선택

보간 방법 선택 기준

01 · 값 변화

서서히 변하는 센서는 보간 중심

02 · 상태 유지

설정값·모드는 ffill 중심

03 · 구간 길이

긴 연속 공백은 제한 보간 또는 제외 검토

실습 2. 보간 방법 비교 (선형·시간)

선형 보간과 시간 보간의 차이를 예시와 실데이터로 비교

목표

선형 보간과 시간 보간의 차이를 작은 예시와 실데이터로 비교

단계

- 세 시점 예시에서 선형과 시간 보간의 가운데 값을 비교
- 정규화 데이터에 두 보간을 적용
- 두 결과가 다른 칸 수를 확인

예상 결과

예시에서 선형은 25, 시간은 15; 실데이터는 등간격이라 선형과 시간이 같음(0개)

긴 누락 구간의 보간 한계

앞뒤 직선이 급등락을 매끈하게 덮는 위험

긴 공백 안의 실제 변화를 확인하기 어려운 상황

일정 길이 이상 구간은 비워 두는 판단

limit으로 채우기 범위 제한

CODE

```
norm = df[["제어출력"]].asfreq("10s")
# limit 없음: 긴 구간까지 모두 채움
full = norm[["제어출력"]].interpolate()
print("limit 없음 NaN:", int(full.isna().sum()))    # 출력: 0
# limit=2: 연속 2칸까지만 채움 (긴 구간은 일부 남김)
limited = norm[["제어출력"]].interpolate(limit=2)
print("limit=2 NaN:", int(limited.isna().sum()))    # 출력: 13
```

limit 계열 옵션 정리

옵션	역할	예시 값
limit	연속으로 채울 최대 칸 수	2
limit_direction	채우는 방향 제어	forward · backward · both
limit_area	채울 영역 제어	inside · outside

짧은 안쪽 공백만 골라 채우기 위한 세밀한 통제

보간을 피해야 할 상황

01 · 급변 구간

고장 전조가 직선 보간에 가려지는
위험

02 · 의미 있는 누락

설비 정지처럼 공백 자체가 정보인
경우

03 · 높은 결측 비율

추정값이 대부분인 센서의 낮은 신
뢰도

실습 3. 긴 누락 구간과 limit

연속 결측 길이를 보고 채우기 범위를 제한해 긴 구간 남기기

목표

연속 결측 길이를 확인하고 채우기 범위를 제한해 긴 구간을 남기기

단계

- 빈 값이 연속으로 이어진 길이들을 구하기
- 제한 없이 보간하면 모두 채워짐을 확인
- 연속 두 칸까지만 채우면 긴 구간은 남음을 확인

예상 결과

연속 결측 길이 10·7·1..., 제한 없으면 0, 두 칸 제한이면 13 남음

실습 4. 두 센서 복원과 전후 비교

두 센서를 시간 보간으로 복원하고 채운 자리를 표시해 비교

목표

두 센서를 시간 보간으로 복원하고 채운 자리를 표시해 전후 비교

단계

- 두 센서의 복원 전 결측 개수를 확인
- 시간 보간으로 두 센서를 함께 복원
- 채운 자리를 표시한 전후 그래프로 비교

예상 결과

복원 전 각 22칸 결측, 복원 후 0; 채운 자리가 그래프에 표시

02

전처리 파이프라인

리샘플링과 결측처리 순서, 품질 점검

전처리 파이프라인의 필요성

여러 전처리 단계를 고정된 순서로 묶는 실행 흐름

일관성·재현성·실수 방지를 동시에 확보

재사용

새 데이터에도 같은 흐름 반복 적용

품질 관리

단계 누락과 순서 오류 감소

리샘플링 전처리 흐름 — 격자 생성

STEP 1

1 시간 인덱스

timestamp 변환 · 정렬 · 인덱스 지정



STEP 2

2 간격 진단

diff로 측정 간격과 기준 주기 확인



STEP 3

3 정규화

asfreq 또는 resample로 일정 격자 생성

리샘플링 전처리 흐름 — 결측 처리

STEP 1

4 결측 확인

isna와 그래프로 개수·패턴 점검



STEP 2

5 채우기

센서 성격에 맞는 보간·복사 적용



STEP 3

6 재확인

남은 NaN과 그래프 품질 점검

단계 순서가 만드는 차이

01

정규화 이후 채우기

시간 격자에서 빈 시점을 명확히 확인

02

채우기 이후 정규화

비어 있는 행이 없어 보간 대상 식별 곤란

빈 칸을 드러낸 뒤 채우는 순서가 전처리 품질의 기준

리샘플링 먼저 vs 보간 먼저

순서	동작	결과
정규화 → 보간	격자 생성 후 빈 칸 채움	일정 간격과 결측 복원
보간 → 정규화	빈 행이 드러나기 전 처리	채우기 실패와 간격 불규칙

보간의 전제는 명확한 시간 격자와 빈 칸 위치

isna로 전처리 결과 검증

CODE

```
norm = df[["제어출력", "소입로온도"]].asfreq("10s")
print("처리 전 결측:\n", norm.isna().sum())
clean = norm.interpolate(method="time")
print("\n처리 후 결측:\n", clean.isna().sum())
# 모두 0이면 의도대로 채워진 것
```


전후 그래프 비교 포인트

01 끊겼던 구간의 자연스러운 연결 여부

02 채운 값이 만드는 계단·점프 왜곡 여부

03 긴 연속 구간의 추정값 표시 여부

전후 그래프 비교 포인트 — 핵심 내용

01 실제 측정값과 보간값의 구분

전처리 품질 점검 체크리스트

점검 항목	확인 내용	통과 기준
시간 간격	일정한 간격 유지	모든 시점 동일 간격
결측 처리	남은 NaN 상태	0개 또는 의도한 수
추세 보존	전후 그래프 비교	왜곡 없는 흐름
신뢰도 표시	긴 보간 구간 기록	추정 구간 명시

전처리 품질 점검 체크리스트 — 비교

점검 항목	확인 내용	통과 기준
전체 품질	결측 비율 수준	과도 시 제외 검토

분석 투입 전 마지막 품질 게이트

정비 판단으로 이어지는 전처리

깨끗한 시계열이 추세 분석과 정비 시점 판단의 근거

시간 간격과 결측 품질을 확보해야 상승 속도와 위험 도달 시점 추정

분석 근거

제어출력·소입로온도 변화 추세를 안정적으로 계산

판단 근거

추정값 비율과 데이터 품질까지 함께 해석

실습 5. 전처리 파이프라인 함수

불러오기·정규화·시간 보간을 한 함수로 묶어 자동화

목표

불러오기·정규화·시간 보간을 한 함수로 묶어 전처리를 자동화

단계

- 파일을 시간 인덱스로 만들고 격자로 정규화하는 함수 작성
- 함수 안에서 시간 보간으로 결측을 채우기
- 처리 전후 결측을 함께 출력해 검증

예상 결과

처리 전 각 22칸, 처리 후 0; 한 번의 호출로 깨끗한 데이터

실습 6. 주기별 복합 데이터 정제

여러 정규화 주기로 정제해 행·결측·보간 결과 비교

목표

여러 정규화 주기로 정제해 행 수·결측·보간 결과를 비교

단계

- 10초·30초·1분 주기로 각각 격자 정규화
- 각 주기의 행 수와 정규화 직후 결측을 세기
- 시간 보간 후 결측이 모두 사라지는지 확인

예상 결과

주기가 길수록 행·결측이 줄고, 보간 후 모두 0

실습 7. 정비 판단 연결 리포트

복원 데이터의 추세로 설비 상태를 분류해 리포트 작성

목표

복원한 데이터의 추세로 설비 상태를 분류해 정비 판단 리포트를 작성

단계

- 복원한 제어출력을 이동평균으로 추세와 변화량 계산
- 변화량 기준으로 정상·주의·정비 검토 상태를 분류
- 추세·변화량·판단·추정 비율을 리포트로 출력

예상 결과

최종 추세 0.8·변화 0.374 → 정비 검토 필요, 추정 비율 11퍼센트